CONTROLE DE FLUXO · ENCONTRO 4 · LIGA PROTOFPGA

Gabarito

Parte A — Atividade em aula

Item	Resposta
1	Aparece `j 0x00400018`. É o endereço de `slt $t3, $t0, $s1`, o teste do laço — a instrução marcada pelo rótulo `laco`.
2	`laco` = **`0x00400018`**, `pula` = **`0x00400034`**, `fim` = **`0x00400040`**. Nenhum deles aparece como instrução: rótulos só dão nome a endereços.
3	O número é 8 (ou `0x00000008`, com a exibição em hexadecimal). É quantas instruções o desvio pula, contando a partir da seguinte: `0x00400020 + 8 × 4 = 0x00400040`, que é `fim`.
4	`lw $t4` executa **8** vezes; o `slt` do teste, **9** (oito verdadeiros e o último, falso, que encerra o laço). Run I/O: `Soma = 41` e `Maior = 9`.
5	Ver a tabela abaixo.
6	Erros mais comuns: prever 8 execuções do teste; achar que `$t0` já vale 1 na primeira parada (o ponto de parada para **antes** do incremento); achar que o maior muda logo na primeira volta.
7	O `slt` do `if`: `slt $t3, $t2, $t4` vira `slt $t3, $t4, $t2`. O Run I/O mostra `Soma = 41` e `Maior = 1` — o texto continua dizendo “Maior”, mas o valor é o menor.
8	Também imprime `Maior = 1`. Com `bne`, o desvio passa a ser tomado quando o `slt` dá 1; o `move` acontece quando ele dá 0, isto é, quando `vetor[i] ≤ maior` — que é o critério do menor. Duas correções diferentes, mesmo efeito: trocar a ordem no `slt` muda a pergunta; trocar `beq` por `bne` muda qual resposta provoca o desvio.
9	`Soma = 1634561916` e `Maior = 1634561875`.
10	A nona volta leu a palavra em **`0x10010020`**, logo depois do vetor. Ali começa o texto `msg1`: os bytes de “Soma”, que lidos como palavra valem `0x616D6F53` = 1 634 561 875. Ele foi somado (41 + 1 634 561 875 = 1 634 561 916) e, por ser maior que 9, virou o maior.
11	Resposta pessoal. Exemplo: “Quem decide é o programa: o processador não sabe onde o vetor acaba e continua lendo enquanto o teste mandar.”

Tabela do item 5 — valores em cada parada, antes do incremento:

Volta	`$t0` (i)	`$t4` (vetor[i])	`$t1` (soma)	`$t2` (maior)
1	0	7	7	7
2	1	3	10	7
3	2	9	19	9
4	3	2	21	9
5	4	5	26	9
6	5	8	34	9
7	6	1	35	9
8	7	6	41	9

Sobre os códigos de máquina deste gabarito

Todos os programas, códigos e saídas foram conferidos no MARS 4.5. Se na sua máquina a coluna Basic mostrar `0x00000008` no lugar de 8, é só a exibição: desmarque *Settings → Values displayed in hexadecimal*.

Parte B — Desafio da semana

Uma solução possível (existem outras igualmente corretas):

```
        .data
vetor:  .word  7, 3, 9, 2, 5, 8, 1, 6
msg:    .asciiz "Maiores que o limite: "

        .text
        .globl main
main:
        li   $v0, 5             # lê o limite do teclado
        syscall
        move $s2, $v0           # $s2 = limite

        la   $s0, vetor         # endereço do elemento atual
        li   $s1, 8             # n
        li   $t0, 0             # i = 0
        li   $t1, 0             # contador = 0

laco:   slt  $t3, $t0, $s1      # i < n ?
        beq  $t3, $zero, fim

        lw   $t4, 0($s0)        # vetor[i]
        slt  $t3, $s2, $t4      # limite < vetor[i] ?
        beq  $t3, $zero, pula  # se não, não conta
        addi $t1, $t1, 1        # contador++
pula:
        addi $t0, $t0, 1        # i++
        addi $s0, $s0, 4
        j    laco

fim:    li   $v0, 4
        la   $a0, msg
        syscall
        li   $v0, 1
        move $a0, $t1
        syscall
        li   $v0, 10
        syscall
```

Resultados conferidos no MARS: limite 5 → `Maiores que o limite: 4` (7, 9, 8 e 6); limite 0 → 8; limite 9 → 0 (9 não é maior que 9).

Pergunta extra. Com limite −1, o resultado é 8: todos os elementos são maiores que −1. Funciona porque `slt` compara com sinal (complemento de dois, encontro 2): −1 é `0xFFFFFFFF`, mas `slt` o trata como −1, não como um número enorme.

O erro mais comum é escrever `slt $t3, $t4, $s2`, que pergunta se o elemento é **menor** que o limite. “vetor[i] > limite” é o mesmo que “limite < vetor[i]”: a ordem no `slt` precisa ser trocada.

Parte C — Exercícios de revisão

1. Um rótulo é um nome dado a um ponto do programa. Na montagem, ele vira o endereço da instrução (ou do dado) que vem logo depois dele. Não gera instrução nenhuma.

2. `j` desvia sempre; `beq` só desvia se os dois registradores forem iguais — senão, o programa segue para a instrução seguinte. Sozinho, quem produz laço infinito é o `j`.

3. (a) 0, porque 5 < 3 é falso. (b) 1. (c) 0: 4 não é menor que 4. (d) 1: a comparação é com sinal, e −7 < 2.

4. Respostas:

```
# (a) vá para L se $s0 >= $s1
slt  $t0, $s0, $s1
beq  $t0, $zero, L

# (b) vá para L se $s0 > $s1  (o mesmo que $s1 < $s0)
slt  $t0, $s1, $s0
bne  $t0, $zero, L
```

5. Porque o desvio serve para sair do caminho. Quando a condição do C é verdadeira, basta seguir em frente para entrar no bloco; só quando ela é falsa é preciso pular. Por isso o desvio é escrito com a condição que faz pular — a negação da condição do C.

6. Uma solução:

```
      bne  $s0, $s1, senao   # a != b -> else
      add  $s2, $s0, $s1      # c = a + b
      j    fimse
senao: sub  $s2, $s0, $s1      # c = a - b
fimse:
```

A condição do C é `a == b`; o desvio que pula o “então” usa a condição invertida, `a != b`, ou seja, `bne`. Conferido no MARS: a = 7, b = 7 → c = 14; a = 7, b = 3 → c = 4. Não é preciso `slt`: igualdade se testa direto.

7. O teste executa **11** vezes (10 verdadeiros e o último, falso); o corpo, **10**; o incremento, **10**.

8. Em binário, `0001 0001 0110 0000 0000 0000 0000 1000`. Reagrupando em 6-5-5-16:

op	rs	rt	imediato
000100	**01011**	**00000**	**0000000000001000**
4 → beq	11 → $t3	0 → $zero	8

É `beq $t3, $zero, …`. O destino é a instrução seguinte mais 8 instruções: `0x0040001C + 4 + 8 × 4 = 0x00400040`. É o desvio para `fim` do programa da aula.

9. Sim. Basta calcular, antes do laço, o endereço logo depois do último elemento — base + 4 · n — e testar se `$s0` ainda é menor que ele:

```
      la   $s0, vetor          # endereço do elemento atual
      addi $s1, $s0, 32        # endereço logo depois do último (8 × 4)
laco: slt  $t3, $s0, $s1       # ainda dentro do vetor?
      beq  $t3, $zero, fim
      lw   $t4, 0($s0)
      …
      addi $s0, $s0, 4
      j    laco
```

Conferido no MARS: a soma continua dando 41, com uma instrução a menos por volta. É a mesma ideia de “andar com o endereço”, levada até o fim.

10. Imprime **128**. O `add` executa **7** vezes: x passa por 2, 4, 8, 16, 32, 64 e 128; na oitava vez que o teste executa, 128 < 100 é falso e o laço termina.

11. Uma solução:

```
        .data
msg:    .asciiz "Soma = "
        .text
        .globl main
main:   li   $v0, 5
        syscall
        move $s0, $v0           # N
        li   $t0, 1             # i = 1
        li   $t1, 0             # soma = 0
laco:   slt  $t2, $s0, $t0      # N < i ?  (ou seja, i > N)
        bne  $t2, $zero, fim    # se i > N, termina
        add  $t1, $t1, $t0      # soma += i
        addi $t0, $t0, 1        # i++
        j    laco
fim:    li   $v0, 4
        la   $a0, msg
        syscall
        li   $v0, 1
        move $a0, $t1
        syscall
        li   $v0, 10
        syscall
```

Conferido no MARS: N = 10 → `Soma = 55`; N = 100 → `Soma = 5050`; N = 0 → `Soma = 0`. O caso N = 0 funciona sozinho porque o teste está no topo: na primeira vez, i = 1 já é maior que N, o desvio sai do laço e o corpo nunca executa. A soma fica com o valor inicial, 0. Em C, o laço é `for (i = 1; i <= N; i++)`. O desvio que sai dele usa a condição invertida, `i > N` — a linha `a > b` da tabela de comparações: `slt` com a ordem trocada e `bne`.